Machine learning Lab Observation

ITA0614 - Machine Learning

Experiment 4

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Name : Variganji Sumanth Babu

Reg.No : 192524447

Problem 1: XOR Logic Gate

Aim

To implement a small feed-forward neural network with backpropagation and learn the XOR truth table, which is not linearly separable.

Algorithm

1. Initialize the network weights and biases to small values.
2. Perform a forward pass: compute hidden-layer activations and the output prediction.
3. Calculate the output error by comparing the prediction with the target value.
4. Propagate the error backward through the network using activation derivatives.
5. Update weights and biases with the learning rate; repeat for the training epochs and evaluate.

Weight update: w(new) = w(old) - η × ∂E/∂w; hidden-layer error uses the next-layer error and activation derivative.

Program:

backprop_a.py - Python 3

```python
# Sub-program 1: Backpropagation on the XOR logic gate (pure Python)
import math, random
random.seed(1)
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]
sigmoid = lambda z: 1 / (1 + math.exp(-z))
W1 = [[random.uniform(-1, 1) for _ in range(2)] for _ in range(2)]
b1, W2, b2 = [0.0, 0.0], [random.uniform(-1, 1) for _ in range(2)], 0.0
for epoch in range(15000):
    for x, target in zip(X, y):
        h = [sigmoid(sum(W1[j][i] * x[i] for i in range(2)) + b1[j]) for j in range(2)]
        out = sigmoid(sum(W2[j] * h[j] for j in range(2)) + b2)
        d2 = (out - target) * out * (1 - out)
        d1 = [d2 * W2[j] * h[j] * (1 - h[j]) for j in range(2)]
        for j in range(2):
            W2[j] -= 0.8 * d2 * h[j]
            for i in range(2): W1[j][i] -= 0.8 * d1[j] * x[i]
            b1[j] -= 0.8 * d1[j]
        b2 -= 0.8 * d2
print("XOR truth table predictions:")
correct = 0
for x, target in zip(X, y):
    h = [sigmoid(sum(W1[j][i] * x[i] for i in range(2)) + b1[j]) for j in range(2)]
    out = sigmoid(sum(W2[j] * h[j] for j in range(2)) + b2)
    pred = int(out >= 0.5); correct += pred == target
    print(x, "target =", target, "output =", round(out, 3), "prediction =", pred)
print("Training accuracy:", f"{correct}/{len(y)}")
```

Input

XOR data set: 4 examples, 2 binary input attributes (A and B), and target XOR (0 / 1). The examples are stored as Python lists. A two-hidden-unit network is trained using sigmoid activation.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Output



Result

The network correctly classifies all four XOR examples (4/4). The hidden layer allows the network to learn the non-linear XOR relationship, unlike a single linear threshold unit.

Problem 2: AND Logic Gate

Aim

To train an Artificial Neural Network with NumPy arrays and backpropagation to learn the AND logic gate.

Algorithm

1. Initialize the network weights and biases to small values.
2. Perform a forward pass: compute hidden-layer activations and the output prediction.
3. Calculate the output error by comparing the prediction with the target value.
4. Propagate the error backward through the network using activation derivatives.
5. Update weights and biases with the learning rate; repeat for the training epochs and evaluate.

Weight update: w(new) = w(old) - η × ∂E/∂w; hidden-layer error uses the next-layer error and activation derivative.

Program:

backprop_b.py - Python 3

```python
# Sub-program 2: Backpropagation on the AND gate using NumPy
import numpy as np
np.random.seed(12)
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[0],[0],[1]], dtype=float)
sigmoid = lambda z: 1 / (1 + np.exp(-z))
W1 = np.random.randn(2, 3) * 0.5; b1 = np.zeros((1, 3))
W2 = np.random.randn(3, 1) * 0.5; b2 = np.zeros((1, 1))
for epoch in range(5000):
    h = sigmoid(X @ W1 + b1); out = sigmoid(h @ W2 + b2)
    d2 = (out - y) * out * (1 - out)
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 -= 1.0 * h.T @ d2 / len(X); b2 -= 1.0 * d2.mean(axis=0, keepdims=True)
    W1 -= 1.0 * X.T @ d1 / len(X); b1 -= 1.0 * d1.mean(axis=0, keepdims=True)
h = sigmoid(X @ W1 + b1); out = sigmoid(h @ W2 + b2)
print("AND truth table predictions:")
pred = (out >= 0.5).astype(int)
for x, target, score, label in zip(X.astype(int), y.astype(int).ravel(), out.ravel(), pred.ravel()):
    print(x.tolist(), "target =", target, "output =", round(float(score), 3), "prediction =", int(label))
print("Training accuracy:", f"{int((pred == y).sum())}/{len(y)}")
```

Input

AND data set: 4 examples, 2 binary inputs (A and B), and target AND (0 / 1). The input and target data are stored in NumPy arrays; the network has a hidden layer of 3 sigmoid units.

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

Output

Terminal - Output

```
$ python backprop_b.py
AND truth table predictions:
[0, 0] target = 0 output = 0.0 prediction = 0
[0, 1] target = 0 output = 0.023 prediction = 0
[1, 0] target = 0 output = 0.024 prediction = 0
[1, 1] target = 1 output = 0.959 prediction = 1
Training accuracy: 4/4
```

Result

The trained NumPy neural network classifies all four AND examples correctly (4/4). The output is close to 0 for the three negative examples and close to 1 for the positive example.

Problem 3: Student Pass Classification

Aim

To apply backpropagation to a small student dataset using study level, attendance level, and assignment completion to predict Pass or Fail.

Algorithm

1. Initialize the network weights and biases to small values.
2. Perform a forward pass: compute hidden-layer activations and the output prediction.
3. Calculate the output error by comparing the prediction with the target value.
4. Propagate the error backward through the network using activation derivatives.
5. Update weights and biases with the learning rate; repeat for the training epochs and evaluate.

Weight update: w(new) = w(old) - η × ∂E/∂w; hidden-layer error uses the next-layer error and activation derivative.

Program:

backprop_c.py - Python 3

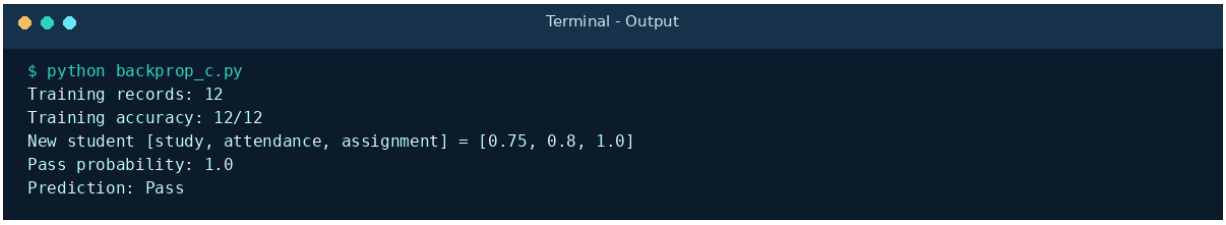
```python
# Sub-program 3: Student pass classification using backpropagation
import numpy as np
np.random.seed(20)
X = np.array([
 [0.2,0.3,0],[0.3,0.5,0],[0.4,0.4,1],[0.5,0.5,0],
 [0.5,0.7,1],[0.6,0.6,1],[0.7,0.5,1],[0.8,0.8,1],
 [0.9,0.7,1],[0.4,0.8,0],[0.7,0.8,0],[0.8,0.4,1]
], dtype=float)
y = np.array([[0],[0],[0],[0],[1],[1],[1],[1],[1],[0],[0],[1]], dtype=float)
sigmoid = lambda z: 1 / (1 + np.exp(-np.clip(z, -40, 40)))
W1 = np.random.randn(3, 5) * 0.7; b1 = np.zeros((1, 5))
W2 = np.random.randn(5, 1) * 0.7; b2 = np.zeros((1, 1))
for epoch in range(6000):
    h = sigmoid(X @ W1 + b1); out = sigmoid(h @ W2 + b2)
    d2 = (out - y) / len(X); d1 = (d2 @ W2.T) * h * (1 - h)
    W2 -= 1.2 * h.T @ d2; b2 -= 1.2 * d2.sum(axis=0, keepdims=True)
    W1 -= 1.2 * X.T @ d1; b1 -= 1.2 * d1.sum(axis=0, keepdims=True)
h = sigmoid(X @ W1 + b1); out = sigmoid(h @ W2 + b2)
pred = (out >= 0.5).astype(int)
print("Training records:", len(X))
print("Training accuracy:", f"{int((pred == y).sum())}/{len(y)}")
new = np.array([[0.75, 0.80, 1.0]])
score = sigmoid(sigmoid(new @ W1 + b1) @ W2 + b2)[0,0]
print("New student [study, attendance, assignment] =", new[0].tolist())
print("Pass probability:", round(float(score), 3))
print("Prediction:", "Pass" if score >= 0.5 else "Fail")
```

Input

Student data set: 12 illustrative examples. Study and Attendance are scaled to the 0-1 range; Assignment is 1 for completed and 0 for pending. The target Pass is encoded as 1 (Pass) or 0 (Fail). The table shows the exact training records used by the program.

Study	Attendance	Assignment	Pass
0.2	0.3	0	0
0.3	0.5	0	0
0.4	0.4	1	0
0.5	0.5	0	0
0.5	0.7	1	1
0.6	0.6	1	1
0.7	0.5	1	1
0.8	0.8	1	1
0.9	0.7	1	1
0.4	0.8	0	0
0.7	0.8	0	0
0.8	0.4	1	1

Output



Result

The network correctly fits the 12 illustrative training records (12/12). A new student with study = 0.75, attendance = 0.80, and assignment completed = 1 is predicted to Pass. This tiny artificial dataset demonstrates the method only; it is not a validated student-assessment tool.

Problem 4: Iris Flower Classification

Aim

To implement multiclass backpropagation with a sigmoid hidden layer and softmax output to classify Iris flowers.

Algorithm

1. Initialize the network weights and biases to small values.
2. Perform a forward pass: compute hidden-layer activations and the output prediction.
3. Calculate the output error by comparing the prediction with the target value.
4. Propagate the error backward through the network using activation derivatives.
5. Update weights and biases with the learning rate; repeat for the training epochs and evaluate.

Weight update: w(new) = w(old) - η × ∂E/∂w; hidden-layer error uses the next-layer error and activation derivative.

Program:

backprop_d.py - Python 3

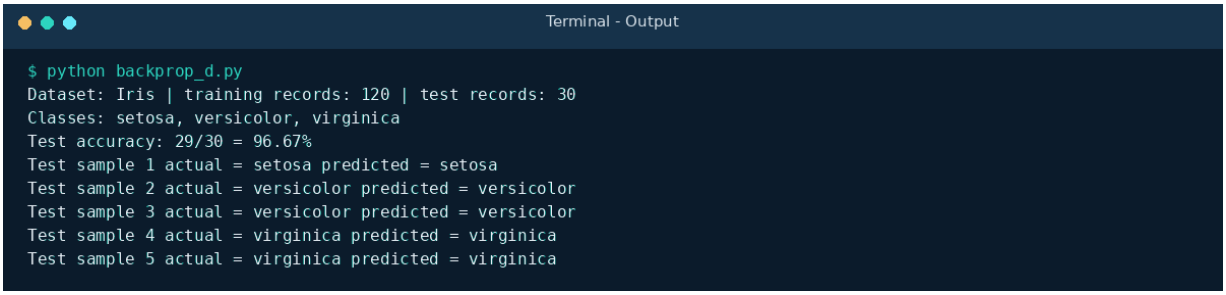
```python
# Sub-program 4: Iris flower classification with a softmax ANN
import numpy as np
from sklearn.datasets import load_iris
np.random.seed(7)
data = load_iris()
X, y = data.data, data.target
order = np.random.permutation(len(y)); train, test = order[:120], order[120:]
lo, hi = X[train].min(axis=0), X[train].max(axis=0)
Xtr = (X[train] - lo) / (hi - lo); Xte = (X[test] - lo) / (hi - lo)
Y = np.eye(3)[y[train]]
sigmoid = lambda z: 1 / (1 + np.exp(-np.clip(z, -40, 40)))
def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True)); return e / e.sum(axis=1, keepdims=True)
W1 = np.random.randn(4, 8) * 0.5; b1 = np.zeros((1, 8))
W2 = np.random.randn(8, 3) * 0.5; b2 = np.zeros((1, 3))
for epoch in range(3500):
    h = sigmoid(Xtr @ W1 + b1); out = softmax(h @ W2 + b2)
    d2 = (out - Y) / len(train); d1 = (d2 @ W2.T) * h * (1 - h)
    W2 -= 1.0 * h.T @ d2; b2 -= 1.0 * d2.sum(axis=0, keepdims=True)
    W1 -= 1.0 * Xtr.T @ d1; b1 -= 1.0 * d1.sum(axis=0, keepdims=True)
hte = sigmoid(Xte @ W1 + b1); test_out = softmax(hte @ W2 + b2)
pred = test_out.argmax(axis=1); acc = (pred == y[test]).mean()
print("Dataset: Iris", "| training records:", len(train), "| test records:", len(test))
print("Classes:", ", ".join(data.target_names))
print("Test accuracy:", f"{int((pred == y[test]).sum())}/{len(test)} = {acc:.2%}")
for i in range(5):
    print("Test sample", i + 1, "actual =", data.target_names[y[test[i]]],
          "predicted =", data.target_names[pred[i]])
```

Input

Iris data set: 150 records, 4 numerical attributes (sepal length, sepal width, petal length, petal width), and 3 classes (setosa, versicolor, virginica). The table shows representative source records only; the program loads all 150 samples from scikit-learn and uses a seeded 120/30 train-test split.

Sepal L.	Sepal W.	Petal L.	Petal W.	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6.0	2.5	virginica
5.8	2.7	5.1	1.9	virginica

Output



Result

The manually implemented softmax neural network classifies 29 of 30 held-out Iris samples correctly (96.67% test accuracy). The displayed sample predictions agree with their labels. The result depends on the random split and training settings.

Problem 5: House Price Estimation

Aim

To use a neural network with backpropagation and a linear output unit to estimate house prices from area and number of bedrooms.

Algorithm

1. Initialize the network weights and biases to small values.
2. Perform a forward pass: compute hidden-layer activations and the output prediction.
3. Calculate the output error by comparing the prediction with the target value.
4. Propagate the error backward through the network using activation derivatives.
5. Update weights and biases with the learning rate; repeat for the training epochs and evaluate.

Weight update: w(new) = w(old) - η × ∂E/∂w; hidden-layer error uses the next-layer error and activation derivative.

Program:

backprop_e.py - Python 3

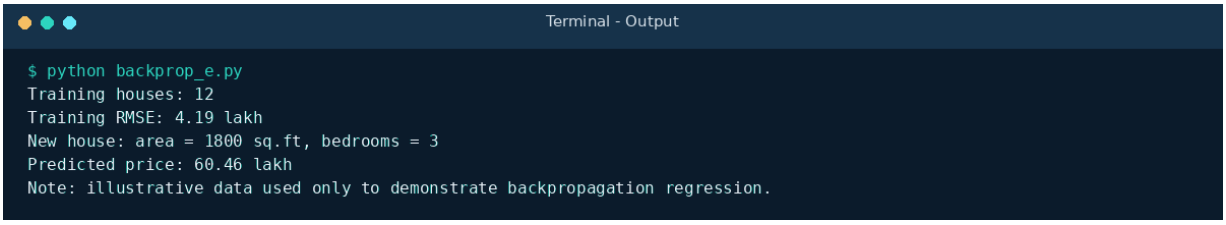
```python
# Sub-program 5: House price estimation with backpropagation regression ANN
import numpy as np
np.random.seed(11)
area = np.array([600,800,1000,1200,1400,1600,1800,2000,2200,2400,2600,2800], dtype=float)
beds = np.array([1,2,2,2,3,3,3,3,4,4,4,4], dtype=float)
price = np.array([20,26,32,38,45,50,56,61,68,73,79,84], dtype=float)
X = np.column_stack((area / 3000, beds / 5)); y = (price / 100).reshape(-1, 1)
sigmoid = lambda z: 1 / (1 + np.exp(-np.clip(z, -40, 40)))
W1 = np.random.randn(2, 5) * 0.5; b1 = np.zeros((1, 5))
W2 = np.random.randn(5, 1) * 0.5; b2 = np.zeros((1, 1))
for epoch in range(8000):
    h = sigmoid(X @ W1 + b1); out = h @ W2 + b2
    d2 = 2 * (out - y) / len(X); d1 = (d2 @ W2.T) * h * (1 - h)
    W2 -= 0.5 * h.T @ d2; b2 -= 0.5 * d2.sum(axis=0, keepdims=True)
    W1 -= 0.5 * X.T @ d1; b1 -= 0.5 * d1.sum(axis=0, keepdims=True)
h = sigmoid(X @ W1 + b1); out = h @ W2 + b2
rmse = float(np.sqrt(np.mean((out - y) ** 2)) * 100)
new = np.array([[1800 / 3000, 3 / 5]])
pred = float((sigmoid(new @ W1 + b1) @ W2 + b2)[0,0] * 100)
print("Training houses:", len(X))
print("Training RMSE:", round(rmse, 2), "lakh")
print("New house: area = 1800 sq.ft, bedrooms = 3")
print("Predicted price:", round(pred, 2), "lakh")
print("Note: illustrative data used only to demonstrate backpropagation regression.")
```

Input

House price data set: 12 illustrative records. Inputs are area (square feet) and bedroom count, scaled before training. The continuous target is price in lakh rupees. The network minimizes mean squared error using backpropagation; the records are synthetic and intended only for demonstration.

Area (sq.ft)	Bedrooms	Price (lakh)
600	1	20
800	2	26
1000	2	32
1200	2	38
1400	3	45
1600	3	50
1800	3	56
2000	3	61
2200	4	68
2400	4	73
2600	4	79
2800	4	84

Output



Result

The regression network obtains a training RMSE of about 4.19 lakh on this small synthetic dataset. For a house with 1800 square feet and 3 bedrooms, it predicts approximately 60.46 lakh. This is a demonstration of neural-network regression, not a real property valuation.